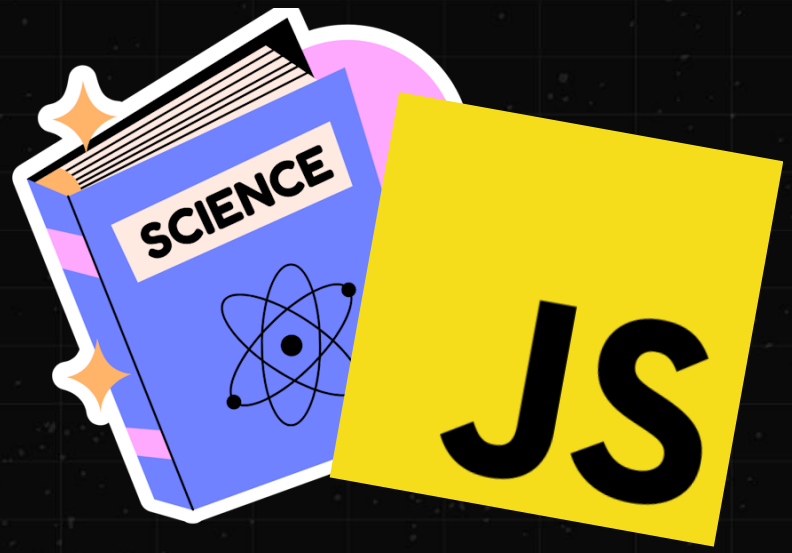
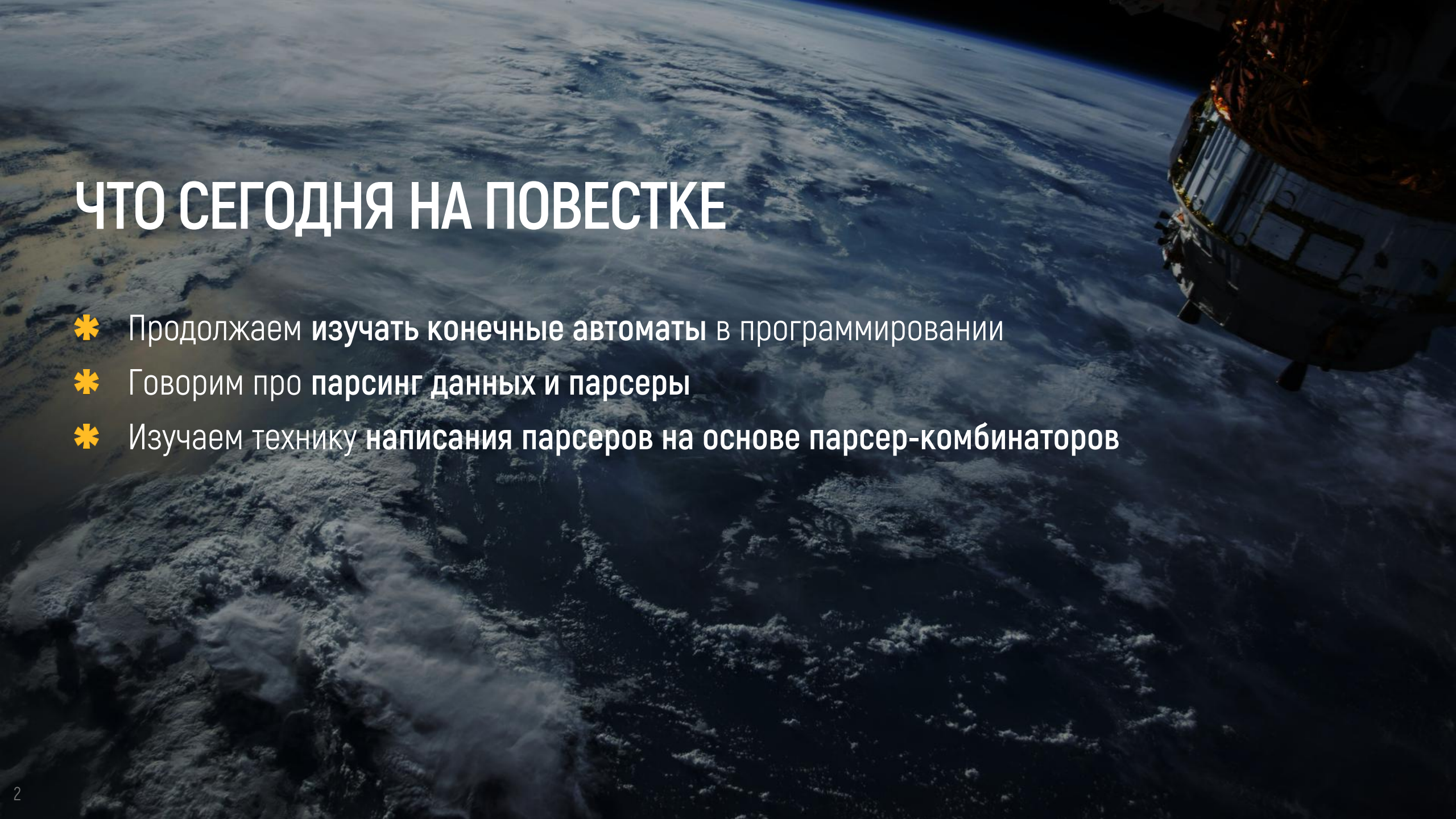
KOBEZZA.LAB

ЛЕКЦИЯ #26

CS BO FRONTEND





ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- Продолжаем **изучать конечные автоматы** в программировании
- Говорим про **парсинг данных и парсеры**
- Изучаем технику **написания парсеров на основе парсер-комбинаторов**

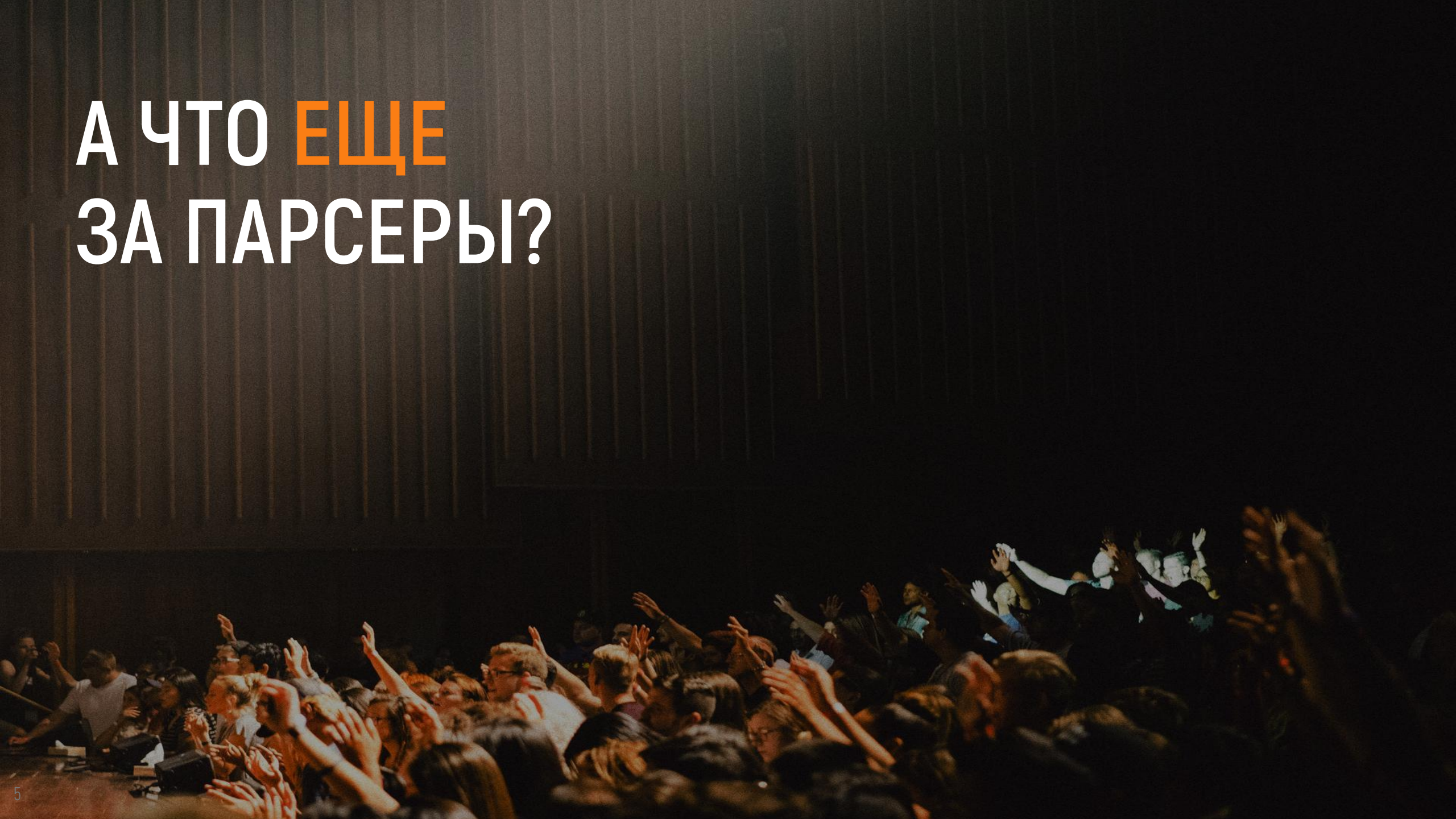
ОПЯТЬ ГОВОРИМ ПРО АВТОМАТЫ?



КА ПОЗВОЛЯЮТ ОПИСЫВАТЬ НАМ

- **Зависимые сущности**
- Например, логика DnD: mousedown ⟶ mousemove | mouseup
- Или написание **парсера синтаксиса некоторого языка**

А ЧТО ЕЩЕ ЗА ПАРСЕРЫ?



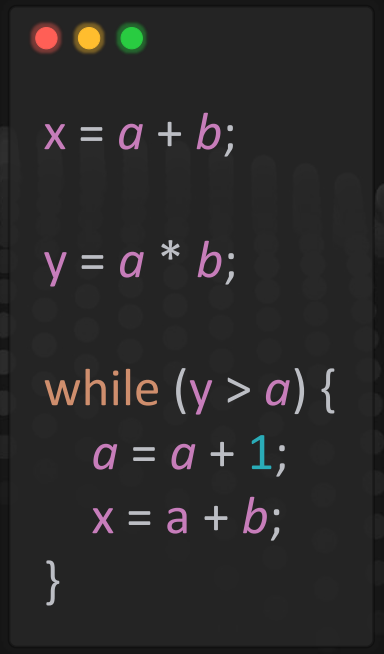
НУ СМОТРИТЕ

- При чтении любой информации из памяти мы фактически **работаем с потоком байт**
- Но нам нужно понимать, **что конкретный байт означает** и как его интерпретировать
- Например, мы хотим написать линтер для некоторого языка программирования
- И вот возникает проблема: нам нужно как-то отобразить наш поток байт на структуры данных, с которыми мы сможем работать
- Программы, **которые делают такое отображение,** называются парсерами

ПАРСИТЬ МОЖНО ЧТО УГОДНО

- Парсер сайта заходит на сайт и извлекает из него информацию
- Парсер капчи извлекает информацию для обхода защиты
- Парсер языка программирования строит абстрактное синтаксическое дерево на основе синтаксиса для дальнейшей работы

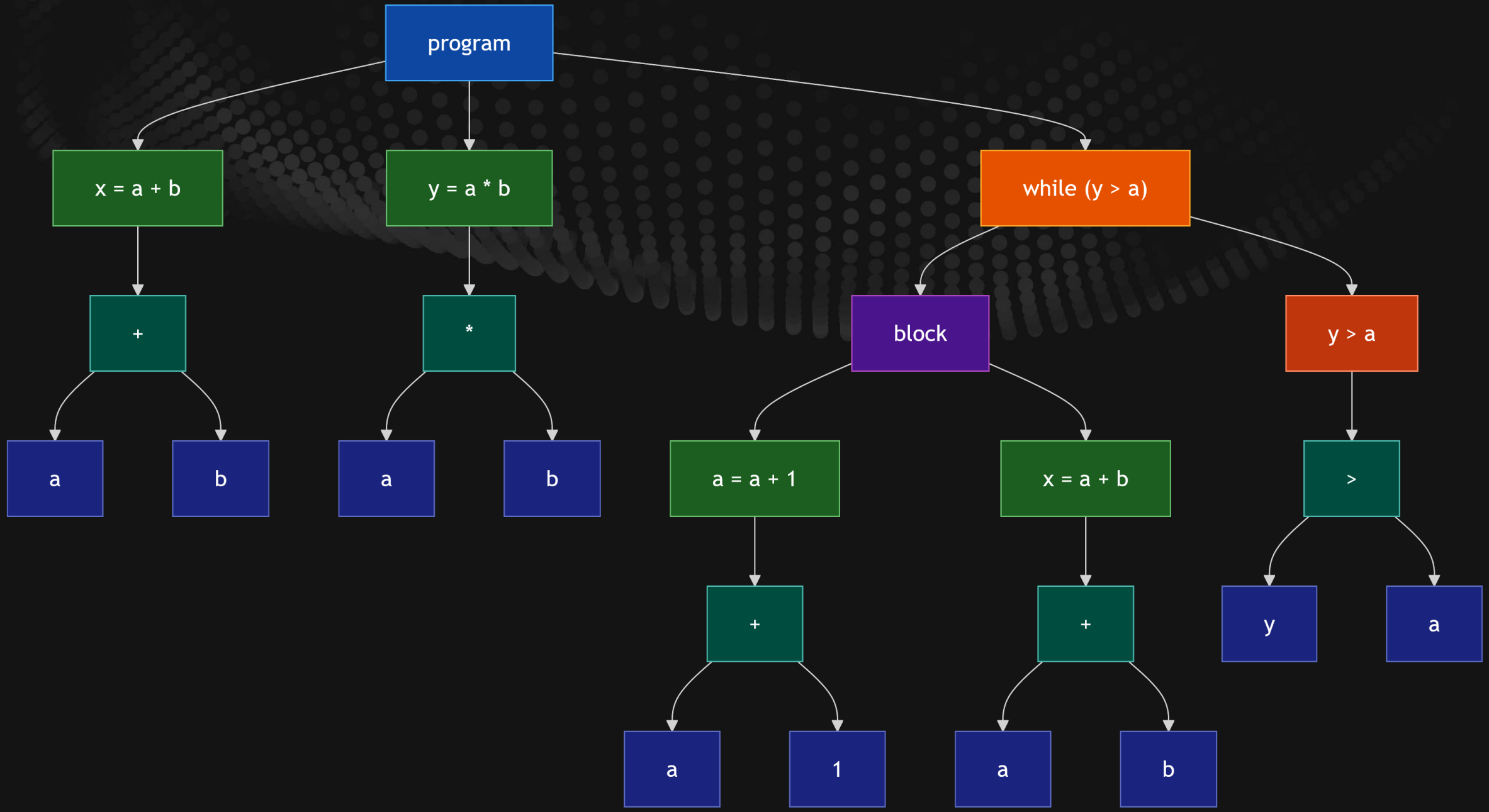
НАПРИМЕР



```
x = a + b;

y = a * b;

while (y > a) {
   a = a + 1;
   x = a + b;
}
```


program
x = a + b
y = a * b
while (y > a)
+
*
block
y > a
a
b
a
b
a = a + 1
x = a + b
>
+
+
y
a
a
1
a
b

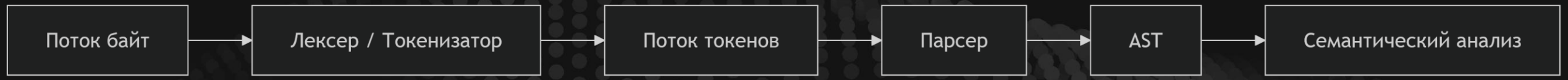
РЕФЛЕКСИРУЕМ

- Используя абстрактное синтаксическое дерево, можно уже работать с этим кодом — **проверять его валидность, видоизменять и трансформировать**
- Работать просто с потоком текста **было бы крайне сложно**

ЭТАПЫ ПАРСИНГА ИЗ ТЕКСТА (ПОТОКА БАЙТ)

- Сначала нужно **обойти текст посимвольно и выделить лексемы**
- Лексема – это просто **некоторый атомарный фрагмент** текста
- Например, `var id = 145;`
- Здесь 5 лексем: `var`, *id*, =, 145 и ;
- **Каждой лексеме** соответствует некоторый токен
- Например, «ключевое слово» или «литерал числа» и т.д.
- На основе потока токенов уже можно построить AST

ВИЗУАЛИЗИРУЕМ




second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

НУ А КАК ВЫ ХОТЕЛИ

- Тема сложная, и ей посвящено множество фундаментальной литературы
- Сегодня мы будем говорить про **написание текстовых парсеров**

НУЖНО ОПИСАТЬ ГРАММАТИКУ ЯЗЫКА

- То есть **описать допустимый алфавит, слова и грамматические правила**
- Описывать грамматику можно по-разному
- Мы будем использовать для этого наш же язык программирования
- И сделаем это на примере чисел с плавающей точкой
- Например, -13.764E-10

```
const sign = tag(/[+-]/);

const digit0 = tag(/[0-9]/);
const digit1 = tag(/[1-9]/);
const digits = seq(digit1, repeat(digit0, { min: 0 }));

const exp = seq(tag(/[eE]/), opt(sign), digits);
const integer = seq(opt(sign), or(tag("0"), digits));

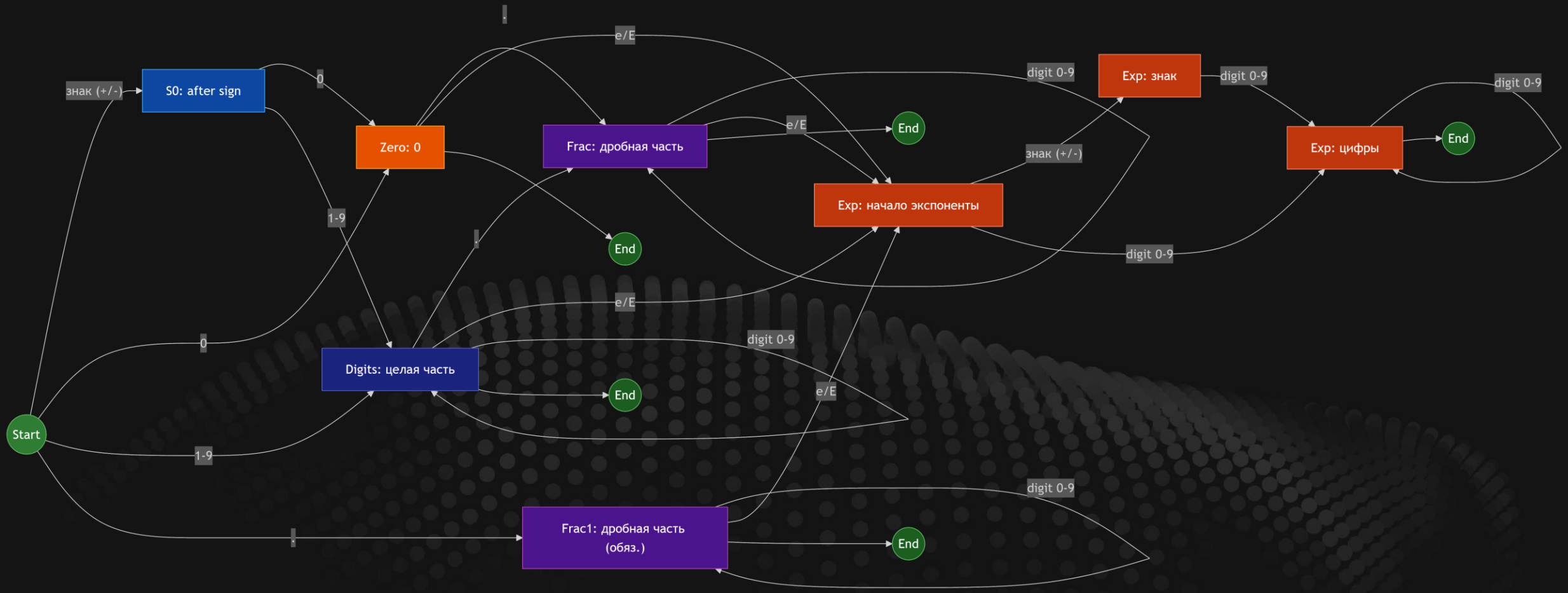
const fractional0 = seq(tag("."), repeat(digit0, { min: 0 }));
const fractional1 = seq(tag("."), repeat(digit0, { min: 1 }));

const floatNumber = or(
    seq(integer, opt(fractional0), opt(exp)),
    seq(fractional1, opt(exp))
);
```


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

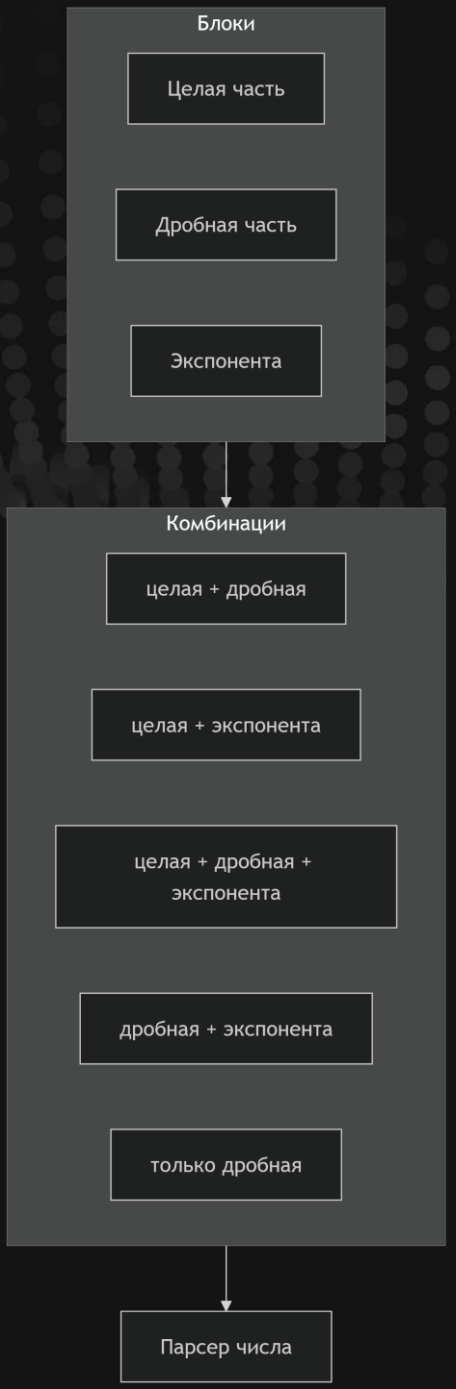
БЕЗ ПАНИКИ

- Мы сейчас одновременно **описали грамматику и автомат**, который будет эту грамматику порождать
- Сейчас мы постепенно разберём этот код, и сложность уйдёт


Start
знак (+/-)
S0: after sign
0
1-9
.
Zero: 0
End
e/E
Frac: дробная часть
End
e/E
digit 0-9
Exp: знак
digit 0-9
Exp: цифры
End
digit 0-9
знак (+/-)
Exp: начало экспоненты
digit 0-9
0
1-9
Digits: целая часть
End
e/E
digit 0-9
.
Frac1: дробная часть
(обяз.)
End
digit 0-9

СЛОЖНОЕ СТРОИТСЯ ИЗ ПРОСТОГО

- Нам нужно описать блоки «целая часть», «дробная часть» и «экспонента»
- А затем описать **все возможные комбинации** этих блоков
- Мы будем делать это с помощью специальных функций-парсеров
- И **выстраивать композиции таких функций** с помощью функций-комбинаторов


Блоки
Целая часть
Дробная часть
Экспонента
Комбинации
целая + дробная
целая + экспонента
целая + дробная + экспонента
дробная + экспонента
только дробная
Парсер числа

ЧТО ЗА ФУНКЦИИ-ПАРСЕРЫ?



ОПИШЕМ ФУНКЦИЮ

- Она будет **принимать некоторый итератор** по исходной строке
- **И возвращать пару:** первый элемент — некоторая распаршенная сущность, а второй — итератор по оставшейся строке
- Это и будет функция-парсер

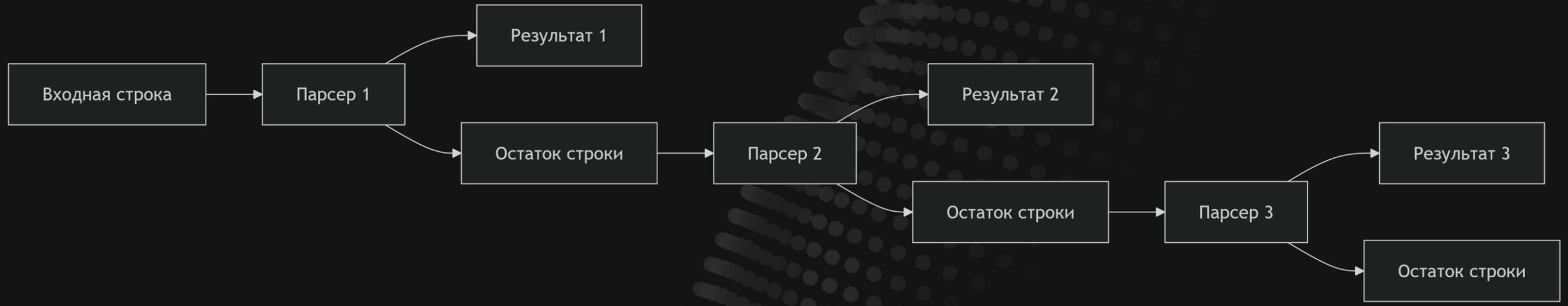
НАПРИМЕР

```ts
export type ParserResult<T> = [T, ParserIterator];

export type Parser<T> = (iter: ParserIterator) => ParserResult<T>;
```

В ЧЕМ ТУТ СОЛЬ

- Каждая функция будет **потреблять некоторую часть строки**
  и обрабатывать её согласно своим правилам
- А в конце давать результат и **итератор по оставшейся строке,**
  чтобы к нему могли подключиться другие итераторы
- **Функции-комбинаторы будут принимать множества таких парсеров**
  и возвращать новый, который будет как-то использовать их вместе

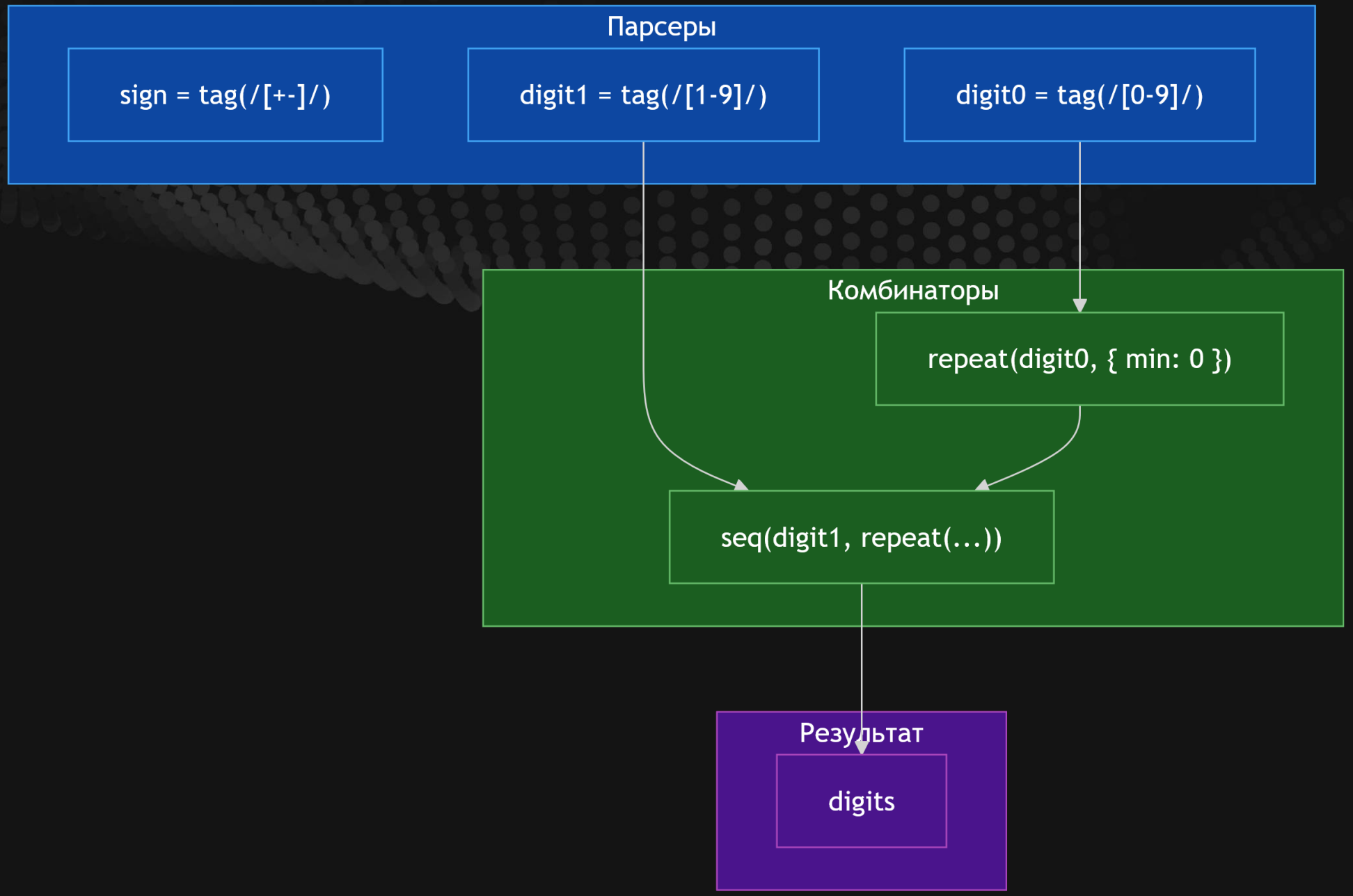

Входная строка
Парсер 1
Результат 1
Остаток строки
Парсер 2
Результат 2
Остаток строки
Парсер 3
Результат 3
Остаток строки

ТЕПЕРЬ МЫ МОЖЕМ ПРОЧИТАТЬ

```
const sign = tag(/[+-]/);

const digit0 = tag(/[0-9]/);
const digit1 = tag(/[1-9]/);

const digits = seq(digit1, repeat(digit0, { min: 0 }));
```


Парсеры
sign = tag(/[+-]/)
digit1 = tag(/[1-9]/)
digit0 = tag(/[0-9]/)
Комбинаторы
repeat(digit0, { min: 0 })
seq(digit1, repeat(...))
Результат
digits

НО КАК ЭТО ЗАПРОГРАММИРОВАТЬ?



ДЛЯ НАЧАЛА СДЕЛАЕМ ИТЕРАТОР

- И добавим ему возможность **«подглядывать» вперёд на один символ** без потребления этого символа итератором
- А также возможность **менять позицию итератора**

```ts
export class ParserIterator {
  readonly input: string;

  get position() { return this.#position; }
  #position: number;

  constructor(input: string, position = 0) {
    this.input = input;
    this.changePosition(position);
  }

  [Symbol.iterator]() { return this; }

  clone() { return new ParserIterator(this.input, this.position); }

  peek() { return this.#getChar(this.position); }

  next() {
    const value = this.#getChar(this.position);

    if (value === undefined) {
      return { value, done: true };
    }

    this.#position += value.length;
    return { value, done: false };
  }

  changePosition(position: number) { this.#position = position; }

  #getChar(index: number): string {
    const str = this.input;
    const code = str.charCodeAt(index);

    // Старший суррогат
    if (code >= 0xD800 && code <= 0xDBFF && index + 1 < str.length) {
      const next = str.charCodeAt(index + 1);

      // Младший суррогат
      if (next >= 0xDC00 && next <= 0xDFFF) {
        return str.slice(index, index + 2);
      }
    }

    return str[index];
  }
}
```

И ТЕПЕРЬ МОЖНО НАПИСАТЬ ПЕРВЫЙ ПАРСЕР

- Сделаем фабрику tag, которая принимает строку,
  регулярное выражение или Iterable с этими данными
- И **возвращает парсер,** который сопоставляет шаблон с данными
  из итератора и, если они есть, возвращает найденную подстроку

```ts
import type { Parser } from "./types.ts";

export function tag(tag: Iterable<string | RegExp> | RegExp): Parser<string> {
  const pattern = tag instanceof RegExp ? [tag] : tag;

  return (iter) => {
    let result = "";

    for (const test of Iterator.from(pattern).flatMap(flat).map(createTest)) {
      const char = iter.peek();

      if (char == null || !test(char)) {
        throw new Error(`Expected pattern "${[...pattern].join("")}"`);
      }

      result += char;
      iter.next();
    }

    return [result, iter];
  }

  function flat(value: string | RegExp): IterableIterator<string | RegExp> {
    return Iterator.from(typeof value === "string" ? value : [value] as any);
  }

  function createTest(value: string | RegExp) {
    return typeof value === "string" ?
      (char: string) => char === value :
      (char: string) => value.test(char);
  }
}
```

И ТЕПЕРЬ ИСПОЛЬЗОВАНИЕ

```
const parser = tag([/\d/, "привет"]);

// [ '5привет', ParserIterator { input: '5привет мир' } ]
console.log(parser(new ParserIterator("5привет мир")));
```

ДВИГАЕМСЯ ДАЛЬШЕ

- Теперь нам нужно написать два комбинатора: seq и repeat
- Первый **будет создавать парсер на основе переданных,**
  как бы «склеивая» их вместе
- Второй же будет создавать **парсер на основе зацикливания переданного**

```ts
import type { Parser } from "./types.ts";

export function seq<T extends Parser<any>[]>(...parsers: T): Parser<{ [K in keyof T]: ReturnType<T[K]>[0] }> {
  return (iter) => {
    const results = [];

    for (const parser of parsers) {
      try {
        const [result, nextIter] = parser(iter.clone());
        iter = nextIter;
        results.push(result);

      } catch (cause) {
        throw new Error("Expected sequence", { cause });
      }
    }

    return [results, iter] as any;
  };
}
```

И ТЕПЕРЬ ИСПОЛЬЗОВАНИЕ

```
const parser1 = tag([/\d/, "привет"]);
const parser2 = tag(" мир");

const parser3 = seq(parser1, parser2);

// [ [ '5привет', ' мир' ], ParserIterator { input: '5привет мир' } ]
console.log(parser3(new ParserIterator("5привет мир")));
```

```ts
import type { Parser } from "./types.ts";

export interface RepeatOptions {
  min?: number;
  max?: number;
}

export function repeat<T>(parser: Parser<T>, { min = 1, max = Infinity }: RepeatOptions = {}): Parser<T[]> {
  return (iter) => {
    const results = [];

    let count = 0;

    while (count < max) {
      try {
        const [result, nextIter] = parser(iter.clone());
        iter = nextIter;
        results.push(result);
        count++;

      } catch {
        if (count >= min) {
          break;
        }

        throw new Error(`Expected at least ${min} matches, got ${count}`);
      }
    }

    return [results, iter];
  };
}
```

И ТЕПЕРЬ ИСПОЛЬЗОВАНИЕ

```
const digit0 = tag(/[0-9]/);
const digit1 = tag(/[1-9]/);
const digits = seq(digit1, repeat(digit0, { min: 0 }));

// [
//   [ '1', [ '0', '2', '9', '4', '2' ] ],
//   ParserIterator { input: '102942 привет мир!' }
// ]
console.log(digits(new ParserIterator("102942 привет мир!")));
```


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ПАЗЛ НАЧИНАЕТ СКЛАДЫВАТЬСЯ

- Мы просто **проектируем автомат** за счёт универсальных функций-парсеров и их комбинаций
- Это похоже на описание регулярного выражения, но в терминах ЯП
- Удивительно, но таким образом можно делать **не только парсеры, а любые автоматы**
- Преимущество такого подхода – простота и универсальность



ЧТО ЕЩЕ ОСТАЛОСЬ СДЕЛАТЬ?

АЛЬТЕРНАЦИЯ И ОПЦИОНАЛЬНОСТЬ

- Опциональность можно сделать просто **фасадом над repeat**
- А **альтернации будут похожи на seq**, но вместо массива результатов будут отдавать результат первого успешного парсера

```ts
import type { Parser } from "./types.ts";

import { repeat } from "./repeat.ts";

export function opt<T>(parser: Parser<T>): Parser<T[]> {
  return repeat(parser, { min: 0, max: 1 });
}
```

```ts
export function or<T extends Parser<any>[]>(
  ...parsers: T
): Parser<T extends (infer R extends Parser<any>)[] ? ReturnType<R> : never> {
  return (iter) => {
    let cause;

    for (const parser of parsers) {
      try {
        return parser(iter.clone());

      } catch (err) {
        cause = err;
      }
    }

    throw new Error("Expected one of the alternatives", { cause });
  }
}
```

```
const sign = tag(/[+-]/);
const digit0 = tag(/[0-9]/);
const digit1 = tag(/[1-9]/);
const digits = seq(digit1, repeat(digit0, { min: 0 }));
const exp = seq(tag(/[eE]/), opt(sign), digits);
const integer = seq(opt(sign), or(tag("0"), digits));
const fractional0 = seq(tag("."), repeat(digit0, { min: 0 }));
const fractional1 = seq(tag("."), repeat(digit0, { min: 1 }));

const floatNumber = or(
  seq(integer, opt(fractional0), opt(exp)),
  seq(fractional1, opt(exp))
);

// [
//   [
//     [ [ '-' ], [ '4', [ '2' ] ] ],
//     [ [ '.', [ '0', '1', '2', '3' ] ] ],
//     [
//       [ 'e', [ '-' ], [ '1', [ '0' ] ] ]
//     ]
//   ],
//   ParserIterator { input: '-42.0123e-10' }
// ]
console.dir(floatNumber(new ParserIterator("-42.0123e-10")), { depth: null });
```

ПОЧТИ УСПЕХ

- Но сейчас **работать с результатом работы итератора** очень неудобно
- Нужно создать комбинатор, который позволит управлять тем, что вернёт парсер

```
export function map<I, O>(
    parser: Parser<I>,
    map: (...args: ParserResult<I>) => ParserResult<O>
): Parser<O> {
    return (iter) => map(...parser(iter));
}
```

```
const sign = map(tag(/[+-]/), (value, iter) => [{ type: "SIGN", value }, iter]);
const digit0 = tag(/[0-9]/);
const digit1 = tag(/[1-9]/);
const digits = map(seq(digit1, repeat(digit0, { min: 0 })), ([first, [...rest]], iter) => [
  { type: "DIGITS", value: Number([first, ...rest].join("")) },
  iter
]);

const exp = map(seq(tag(/[eE]/), opt(sign), digits), ([_, sign, digits], iter) => [
  { type: "EXP", value: [...sign, digits] },
  iter
]);

const integer = map(seq(opt(sign), or(tag("0"), digits)), ([sign, digits], iter) => [
  { type: "INTEGER", value: [...sign, digits] },
  iter
]);

const fractional0 = map(seq(tag("."), repeat(digit0, { min: 0 })), ([_, [...digits]], iter) => [
  { type: "FRACTIONAL", value: Number("0." + digits.join("")) },
  iter
]);

const fractional1 = map(seq(tag("."), repeat(digit0, { min: 1 })), ([_, [...digits]], iter) => [
  { type: "FRACTIONAL", value: Number("0." + digits.join("")) },
  iter
]);

const floatNumber = or(
  seq(integer, opt(fractional0), opt(exp)),
  seq(fractional1, opt(exp))
);

// [
//   [
//     {
//       type: 'INTEGER',
//       value: [ { type: 'SIGN', value: '-' }, { type: 'DIGITS', value: 42 } ]
//     },
//     [ { type: 'FRACTIONAL', value: 0.0123 } ],
//     [
//       {
//         type: 'EXP',
//         value: [ { type: 'SIGN', value: '-' }, { type: 'DIGITS', value: 10 } ]
//       }
//     ]
//   ],
//   ParserIterator { input: '-42.0123e-10' }
// ]
console.dir(floatNumber(new ParserIterator("-42.0123e-10")), { depth: null });
```

РЕФЛЕКСИРУЕМ

- Комбинатор map позволил **возвращать из итератора не просто лексемы,** а распаршенные токены
- Используя этот комбинатор, можно строить очень гибкие конструкции


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ВЫДЫХАЕМ

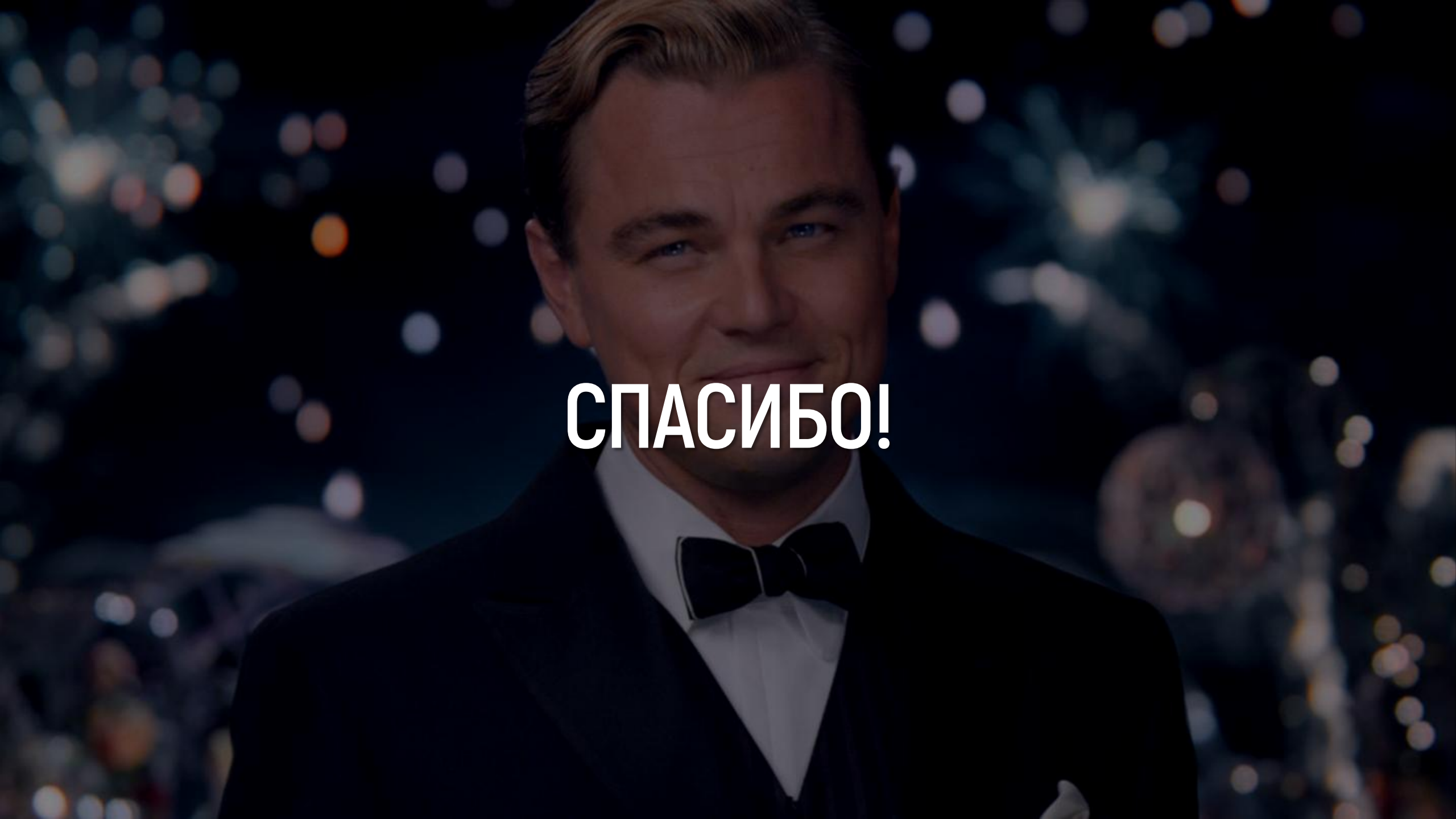
- Написание парсеров как **композиция из маленьких функций** – это невероятно удобный и гибкий подход
- Многие «большие» компиляторы используют такой подход
- Овладение этой простой техникой позволит вам быстро **писать парсеры весьма сложных грамматик**
- А также **масштабировать этот подход** за пределы парсеров

ПОДДЕРЖКА РЕКУРСИИ

- Так как наши парсеры – это обычные функции,
  мы можем легко **сделать рекурсивный парсинг**
- Это необходимое требование для парсинга многих языков
- Например, того же JSON

ПОДВЕДЁМ ИТОГИ

- Парсинг – **процесс превращения «сырых» данных** в понятную структуру: лексемы ⟶ токены ⟶ дерево (AST)
- Комбинаторы позволяют **даже «страшные грамматики»** спокойно собирать из простых функций-парсеров, как из кубиков
- В основе парсеров **всё те же автоматы**, а значит, освоенные техники можно масштабировать и за пределы парсеров
- Например, для программирования логики NPC в играх


СПАСИБО!